Ad Launch + Tombstone OS — Master Architecture Reference

Last updated: April 2026
Maintainer: Tom @ Blazin Hog / Launch Marketing
Deployments:
- Frontend: `connect.launchmarketing.com` (custom domain) / `ad-launch-1nfyr8.abacusai.app`
- Backend (Tombstone): Render-hosted FastAPI service (env var `TOMBSTONE_API_URL`)

Table of Contents

---

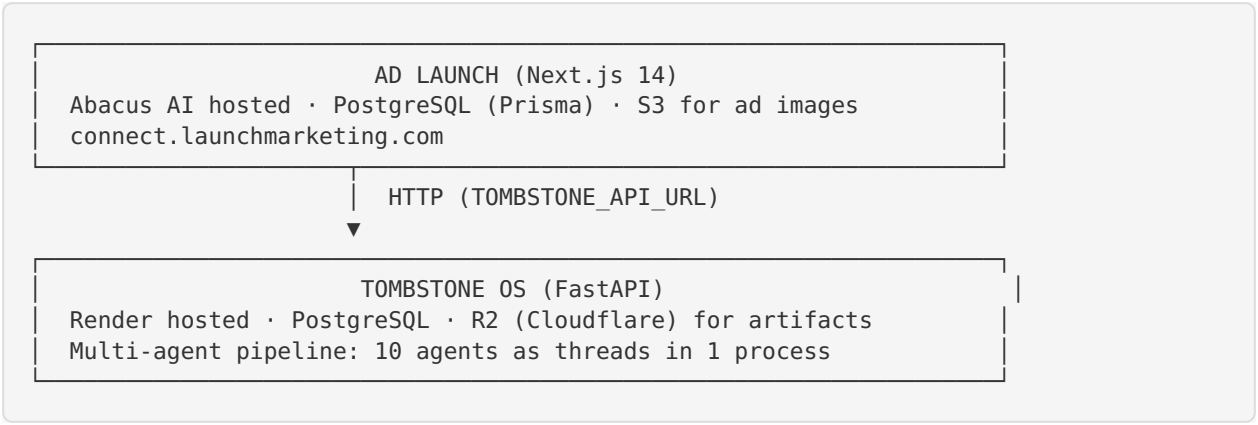
1. System Overview

Ad Launch is a lead-gen platform that takes a business URL, performs deep research via **Tombstone OS** (a multi-agent AI backend), and generates:

- **3 Facebook ad creatives** (one per “lane”: Website/Brand, Local News, Upcoming Holiday)
- **9 social media posts** (3 per lane, same lane taxonomy)

- **Live SEO audit** of the business website
- **Google Ads copy** recommendations
- **90-day posting plan** with holiday/event calendar

The system has two completely separate codebases that communicate via HTTP:



---

2. High-Level Call Flow

Ad Generation (Primary Flow)

```
User enters URL on landing page
        │
        ▼
POST /api/analyze
  ├─ Fetches website HTML (handles Cloudflare 403 gracefully)
  ├─ Extracts business address (4-layer scraper)
  ├─ Falls back to Google Places if scraping fails
  ├─ Upserts Business record
  └─ Creates Analysis (status: pending_location)
        │
        ▼
Frontend shows LocationConfirmCard
  └─ User confirms/edits address
        │
        ▼
POST /api/analysis/[id]/confirm-and-launch
  ├─ Saves confirmed location to Analysis + Business
  ├─ Fires Clark Kent scout (background, fire-and-forget)
  │     └─ POST /api/rss/clark-kent → gathers local RSS + events
  │           └─ POST TOMBSTONE /commands (createSocialMissions)
  │                 → Tombstone creates social workflow
  │                 → socialMissionId stored on Analysis
  ├─ POST TOMBSTONE /commands (createMissions) for ad generation
  │     → Tombstone creates ad workflow
  │     → missionId stored on Analysis (JSON: {website: wfId, news: wfId, holiday: wf
Id})
  └─ Analysis status → "processing"
        │
        ▼
Frontend polls GET /api/mission-status?analysisId=xxx (every 3s)
  ├─ Calls TOMBSTONE /tasks?workflow_ids=... for progress
  ├─ When all workflows complete:
  │     ├─ Calls TOMBSTONE /tasks/[id]/outputs for each workflow
  │     ├─ Calls generateAllAdImages() → GPT-5.1 Designer Brief → S3 upload
  │     ├─ Creates Ad records (one per lane) with S3 URLs
  │     ├─ Runs live SEO audit
  │     └─ Analysis status → "completed"
  └─ Returns step-by-step progress to frontend
        │
        ▼
Results page (/results/[id])
  ├─ 3 ad previews as Facebook post mockups (one per lane)
  ├─ SEO insights (collapsible)
  ├─ Google Ads copy
  └─ 90-day posting plan
```

Social Post Pipeline

```
Clark Kent Scout fires during confirm-and-launch (above)
        ||
        ▼
Tombstone receives social mission
  ├─ Agent chain: Bridger → Zig → Ogilvy → Draper → Warhol
  │  (same chain as ads, but with social-optimized prompts)
        ||
        ▼
Frontend polls GET /api/social/missions/poll?analysisId=xxx
  ├─ Calls TOMBSTONE /content/queue?workflow_ids=...
  ├─ For each completed task, calls TOMBSTONE /content/[taskId]
  ├─ Enriches with caption, hashtags, CTA from upstream siblings
  ├─ Resolves R2 image keys → /api/social/image-proxy proxy URLs
  └─ Creates SocialPost records in Ad Launch DB
        ||
        ▼
Social Posts page (/dashboard/social)
  ├─ Post queue with image display, filter by status
  ├─ Approve / Reject / Edit / Delete / Copy Caption / Download
  └─ Accounts tab for linking social platforms
```

---

3. Ad Launch (Frontend)

3.1 Pages

Path	Purpose
`/`	Landing page — “Social Posting Factory” with SSE demo grid
`/login`	Email/password login
`/confirm`	Email confirmation handler
`/reset-password`	Password reset flow
`/search`	Business search (Google Places powered)
`/analyze/[id]`	Analysis tracker — real-time progress, location confirm, results
`/results/[id]`	Final results — ad previews, SEO, Google Ads copy, posting plan
`/dashboard`	Business cards dashboard — shows all user businesses
`/dashboard/social`	Social post queue — approve/reject/download posts
`/dashboard/social/publishing`	Publish queue — **Coming Soon overlay** (auto-posting not yet live)
`/dashboard/feeds`	Content feeds preferences (national RSS opt-in)
`/admin/rss`	RSS admin dashboard (feeds, items, policies, export)
`/test-ads`	Internal A/B test lab for ad generation

3.2 API Routes

Auth & User

Route	Method	Purpose
`/api/auth/[...nextauth]`	GET/POST	NextAuth handler (credentials provider)
`/api/register`	POST	New user registration (business email only)
`/api/signup`	POST	Alias for registration
`/api/auth/login`	POST	Direct login endpoint
`/api/auth/forgot-password`	POST	Send password reset email
`/api/auth/reset-password`	POST	Process password reset token
`/api/confirm-email`	GET	Email confirmation via token

Analysis Pipeline

Route	Method	Purpose
`/api/analyze`	POST	Create analysis — fetch site, extract address, Google Places fallback
`/api/analysis/[id]`	GET	Fetch analysis with resolved image URLs
`/api/analysis/[id]/confirm-location`	PATCH	Confirm/edit business location
`/api/analysis/[id]/confirm-and-launch`	POST	Save location → fire Clark Kent → launch Tombstone mission
`/api/analysis/[id]/generate-more`	POST	Generate additional ads for a lane
`/api/mission-status`	GET	Poll Tombstone for workflow progress, create ads on completion

Social Content

Route	Method	Purpose
`/api/social/posts`	GET/POST	List/create social posts
`/api/social/posts/[id]`	GET/PATCH/DELETE	Single post CRUD + status workflow
`/api/social/generate`	POST	Manual social post generation via Tombstone
`/api/social/missions/poll`	GET	Poll Tombstone content queue → import SocialPosts
`/api/social/image-proxy`	GET	Server-side proxy for R2 presigned URLs
`/api/social/accounts`	GET/POST/DELETE	Link/unlink social platform accounts

Content / Publishing (Tombstone Proxy)

Route	Method	Purpose
`/api/content/queue`	GET	Proxy to Tombstone `/content/queue` with image resolution
`/api/content/[taskId]`	GET	Proxy to Tombstone `/content/{taskId}` for detail
`/api/publish/[taskId]`	POST	Proxy to Tombstone `/publish/{taskId}`
`/api/publish/accounts`	GET	Proxy to Tombstone `/social/accounts`

RSS Intelligence

Route	Method	Purpose
`/api/rss/clark-kent`	POST	Clark Kent scout — gathers local RSS + events → sends to Tombstone
`/api/rss/trade-area`	GET/POST	Query trade area items by ZIP + radius
`/api/rss/admin/stats`	GET	Dashboard overview stats
`/api/rss/admin/feeds`	GET/POST	Feed list + create
`/api/rss/admin/feeds/[id]`	GET/PATCH/DELETE	Single feed CRUD
`/api/rss/admin/items`	GET/PATCH	Item list + audit override
`/api/rss/admin/policies`	GET/PATCH	Content policy management
`/api/rss/admin/export`	GET/POST	Bulk export (CSV/JSON) + Clark Kent webhook

Misc

Route	Method	Purpose
`/api/resolve-image`	GET	Resolve R2 artifact keys to presigned URLs
`/api/edit-ad`	POST	Regenerate ad image with GPT-5.1 + user instructions
`/api/generate-concept-site`	POST	Generate concept website HTML
`/api/search-businesses`	POST	Google Places business search
`/api/user/businesses`	GET	List user’s businesses with analysis counts
`/api/user/analyses`	GET	List user’s analyses
`/api/user/feed-preferences`	GET/POST	RSS feed preferences (national feed opt-in)
`/api/test-ad-gen`	POST	Internal test endpoint for ad generation
`/api/upgrade-ad-images`	POST	Batch upgrade legacy ad images to GPT-5.1

3.3 Library Modules

Module	Purpose
`lib/tombstone.ts`	Tombstone API client — `createMissions()`, `createSocialMissions()`, `getMultiWorkflowStatus()`, `getWorkflowResults()`, task labels
`lib/generate-ad-image.ts`	GPT-5.1 “Designer Brief” image generation + S3 upload
`lib/aws-config.ts`	S3 client configuration (AWS SDK v3)
`lib/google-places.ts`	Google Places Text Search API wrapper
`lib/seo-audit.ts`	Live lightweight SEO audit (15+ checks, score 0-100)
`lib/address-extractor.ts`	4-layer business address scraper (Schema.org, tags, footer, regex)
`lib/ghl.ts`	GoHighLevel CRM API (contact creation, email sending)
`lib/email-validation.ts`	Business email validation (blocks Gmail, Yahoo, etc.)
`lib/auth-options.ts`	NextAuth configuration (credentials provider, JWT)
`lib/db.ts`	Prisma client singleton
`lib/types.ts`	Shared TypeScript types
`lib/utils.ts`	General utilities

RSS Subsystem (`lib/rss/`)

Module	Purpose
`trade-area-feed.ts`	ZIP → trade area → feeds → items query engine
`content-policy.ts`	Multi-layer keyword content filter
`geo-lookup.ts`	ZIP ↔ city ↔ county ↔ state resolution
`discovery.ts`	RSS feed auto-discovery from websites
`feed-parser.ts`	RSS/Atom feed parsing
`freshness-scorer.ts`	Feed freshness scoring (0-100)
`dedup.ts`	SimHash cross-feed deduplication
`geo-tagger.ts`	Assign geographic coverage to feeds
`source-classifier.ts`	Classify feed type (news, gov, weather, etc.)
`types.ts`	RSS type definitions

Social Subsystem (`lib/social/`)

Module	Purpose
`upcoming-events.ts`	Holiday/event calendar for social content

---

4. Tombstone OS (Backend)

4.1 Agent Roster

Tombstone runs all agents as **threads inside a single Python process** (via `run/thread_runner.py`). The thread runner monkey-patches `task_reliability` so that `complete_task()` instantly signals the downstream department's threading Event — eliminating inter-step polling latency.

Pipeline Agents (claim tasks from department queue)

Agent	Department	Role	Upstream Input	Output
Jim Bridger	Research	Website recon — compliance check, asset discovery, brand palette, business summary	Raw URL from command	`business_summary`, `brand_voice`, `semantic_truth`, `brand_palette`, `offers`
Zig Ziglar	Marketing	Marketing strategy — audience, angles, keyword targeting	Bridger's recon	Marketing strategy, audience personas, messaging framework
David Ogilvy	Creative Strategy	Ad copywriting — multi-ad mode: one headline/body/CTA per angle	Zig's strategy	`ads[]` array with `angle`, `headline`, `body_copy`, `cta` per ad
Don Draper	Creative Direction	Visual direction — art direction, image prompts, campaign concepts	Ogilvy's copy	`campaigns[]` with visual direction, image prompts, color direction
Andy Warhol	Render Production	Image generation — GPT-5.1 full-composition mode (text baked into image)	Draper's campaigns	`renders[]` with image URLs/R2 keys, `full_composition` flag
Peter Drucker	Strategy & Intelligence	Business strategy, competitive analysis	Varies	Strategy recommendations
George Boole	Code Execution	Code generation/execution tasks	Varies	Code output

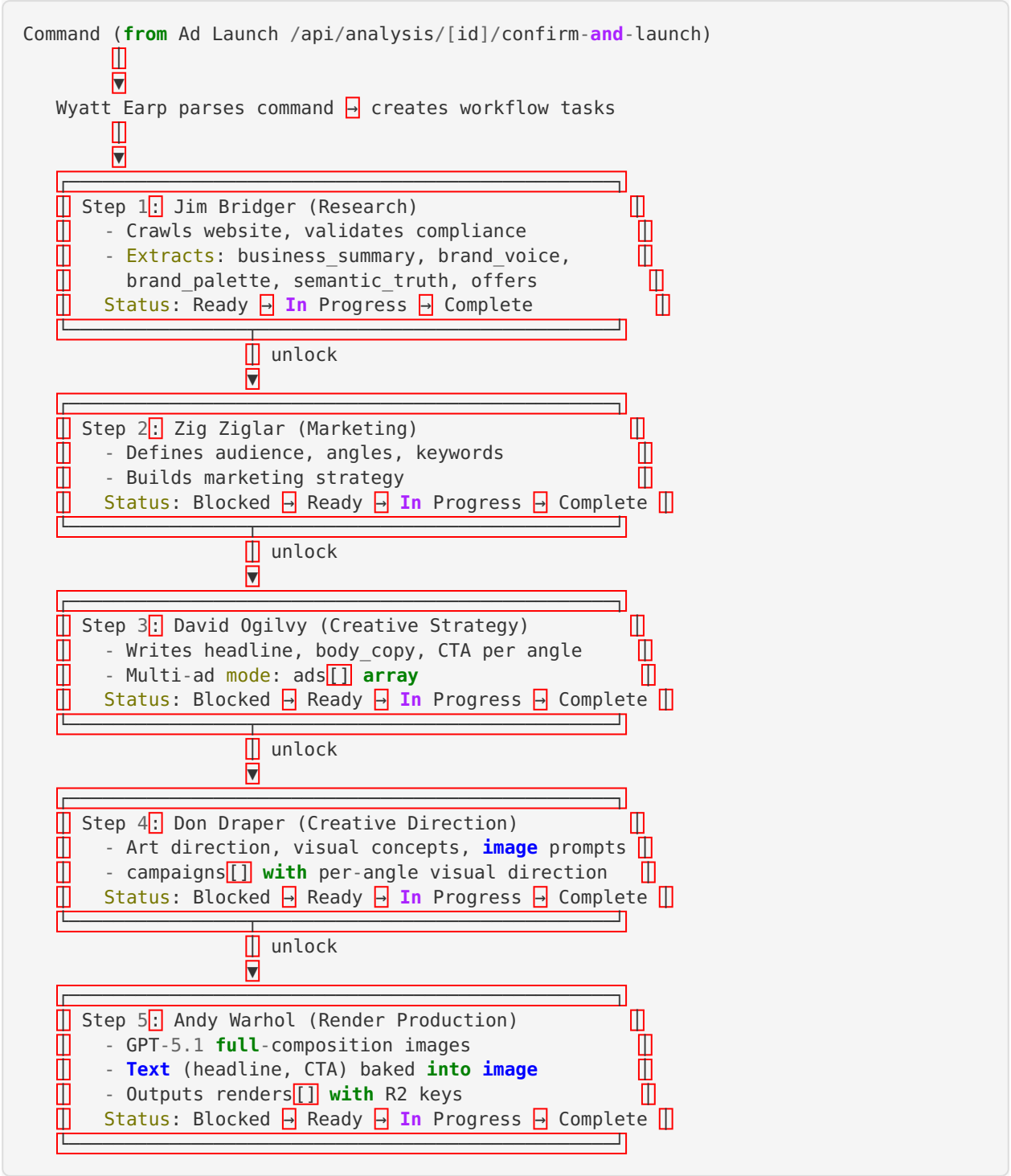
Service Agents (own internal loops, not task-claim based)

Agent	Department	Role
Wyatt Earp	Executive Command	Command router — parses natural language commands, creates missions and work-flow tasks, routes to correct departments
Dispatcher	Operations	Monitors in-progress tasks, registers watchdog entries
Task Watchdog	Operations	Monitors task timeouts — first-update deadline (15 min), hard timeout (60 min), escalation

Inactive / Legacy Agents

Agent	Status	Notes
Claude Hopkins	Inactive	Was “Conversion Assembly” — text overlay compositing. Removed from pipeline when Andy Warhol gained full-composition mode (text baked into generated image). Code still exists in `agents/claude_hopkins.py`.
Allan Pinkerton	Legacy	Security/audit agent. Class-based, not in active roster.
Grace Hopper	Legacy	Class-based agent, not in active thread runner.
Tom Hopkins	Legacy	Sales Coaching department. Has `run_once()` but not in thread runner.
Ada Lovelace	Legacy	Development Architecture. Has `run_once()` but not in thread runner.
Operations Worker	Legacy	General Operations department worker. Not in thread runner.
Research Agent	Legacy	Generic research. Superseded by Jim Bridger.

4.2 Agent Pipeline — Ad Generation



Task Lifecycle: `Ready for Pickup` → (agent claims) → `In Progress` → `Complete` / `Failed`
Blocked tasks: Start as `Blocked`, unblocked by `_unlock_dependent_tasks_internal()` when dependency completes.
Claim system: Each claim gets a UUID `claim_token`. Only the holder can complete/fail the task. Prevents double-processing.
Heartbeats: Agents send heartbeats while working. Watchdog escalates if no heartbeat within deadline.

4.3 Agent Pipeline — Social Content

Same 5-step chain (Bridger → Zig → Ogilvy → Draper → Warhol) but with:
- Social-optimized prompts (caption + hashtags vs ad copy)
- 3 lanes × 3 posts = 9 total tasks
- Clark Kent’s local scout brief embedded in the command
- Bridger independently scouts the website (business identity)

4.4 Core Infrastructure (`core/`)

Module	Purpose
`task_service.py`	Mission/task/workflow CRUD — `create_mission()`, `create_task()`, `create_workflow_tasks()`, `get_task_input_context()`, `unlock_dependent_tasks()`
`task_reliability.py`	Claim tokens, heartbeats, orphan recovery, graceful shutdown, retry-on-failure, `adaptive_sleep()`
`task_contracts.py`	Output schema validation for agents
`model_router.py`	LLM routing — OpenAI (GPT-5.1 default) or Ollama (local, qwen2.5:7b). `route_model_call()`
`pg_runtime.py`	PostgreSQL connection management
`r2_storage.py`	Cloudflare R2 storage client (artifact upload/download/presigned URLs)
`status_service.py`	Task/mission status formatting
`memory_service.py`	Agent memory retrieval for context
`mission_naming.py`	Generates creative mission names
`wyatt_router.py`	Command intent parsing — routes owner messages to correct action
`brand_palette.py`	Brand color extraction utilities
`url_compliance.py`	Website compliance validation (ToS, page count limits — max 500 pages)
`policy_engine.py`	Content policy enforcement
`security_layer.py`	Security utilities
`usage_tracker.py`	LLM usage tracking
`utils.py`	`normalize_payload()`, `ensure_dict_output()`

4.5 Server API — Mission Control (`server/mission_control_api.py`)

FastAPI application. This is what Ad Launch calls via `TOMBSTONE_API_URL`.

Key Endpoints

Endpoint	Method	Purpose
`/commands`	POST	**Primary entry point** — natural language command → Wyatt routes to mission/workflow creation
`/tasks`	GET	List all tasks
`/tasks/{task_id}`	GET	Get single task
`/tasks/{task_id}/outputs`	GET	Get task outputs
`/tasks/{task_id}/reset`	POST	Reset task (re-queue for retry)
`/tasks/{task_id}/artifact`	GET	Get task artifact (resolved file/image)
`/tasks/{task_id}/thumbnail`	GET	Generate thumbnail URL for task
`/tasks/artifact-file`	GET	Raw artifact file access
`/artifacts/resolve`	GET	**Critical** — Resolve R2 artifact key to presigned URL
`/content/queue`	GET	Content queue — filterable by `workflow_ids`, enriched with caption/preview
`/content/{task_id}`	GET	Content detail — `base_caption`, `cta`, `hashtags`, image URLs
`/content/{task_id}`	PUT	Update content draft (user edits)
`/publish/{task_id}`	POST	Publish content to social platforms
`/social/accounts`	GET	List connected social accounts
`/agents`	GET	List all agents with metadata
`/agents/status`	GET	Agent heartbeat status

Endpoint	Method	Purpose
`/agents/{name}/start`	POST	Start an agent process
`/agents/{name}/kill`	POST	Kill an agent process
`/admin/tasks/{task_id}/force-fail`	POST	Force-fail a stuck task
`/uploads`	POST	Upload file to R2
`/health`	GET	Health check
`/ws`	WebSocket	Real-time task/output updates
`/`	GET	Mission control dashboard (HTML)
`/mission-control-v3`	GET	V3 dashboard

4.6 Services

Social Publishers (`services/social_publishers/`)

Stub publishers — no real API calls yet. Used by `publish_worker.py`.

Module	Platform	Status
`facebook.py`	Facebook	Stub
`instagram.py`	Instagram	Stub
`x.py`	X/Twitter	Stub
`linkedin.py`	LinkedIn	Stub

Publish Worker (`workers/publish_worker.py`)

Independent loop (not part of main agent pipeline):
- Polls `scheduled_posts` table every 30 seconds
- Finds rows with `status='pending'` and `scheduled_time <= NOW()`
- Dispatches to platform-specific publisher from `PUBLISHER_REGISTRY`
- Marks rows as `posted` or `failed`
- Uses `FOR UPDATE SKIP LOCKED` for safe concurrency

---

5. Data Models

5.1 Ad Launch (Prisma/PostgreSQL)

Core Business Models

Model	Key Fields	Purpose
User	`id`, `email`, `password`, `confirmed`, `freeAdsUsed`, `role`	User accounts (business email only)
Business	`id`, `userId`, `websiteUrl`, `businessName/Addr/City/State/Zip/Phone`	One per user+URL (`@@unique([userId, websiteUrl])`)
Analysis	`id`, `userId`, `businessId`, `websiteUrl`, `missionId`, `socialMissionId`, `status`, `results`, `seoData`, `postingPlan`, geo fields	Analysis runs — links to Business, stores Tombstone workflow IDs
Ad	`id`, `analysisId`, `imageUrl`, `caption`, `headline`, `lane`, `watermarked`	Generated ads (one per lane: website/news/holiday)

Social Models

Model	Key Fields	Purpose
SocialPost	`id`, `userId`, `analysisId`, `caption`, `hashtags[]`, `imageUrl`, `platforms[]`, `status`, `postType`	Social post queue (draft → pending_approval → approved → published)
SocialAccount	`id`, `userId`, `platform`, `handle`, `profileUrl`	Linked social accounts (`@@unique([userId, platform])`)

RSS Intelligence Models

Model	Key Fields	Purpose
RssFeed	`id`, `url`, `sourceType`, `status`, `geoScope`, `industry`	RSS feed registry (462 feeds, 155 active)
RssItem	`id`, `feedId`, `guid`, `title`, `filterStatus`, `pubDate`	Individual articles (2,360 items)
FeedGeo	`feedId`, `zipId`, `coverageType`, `confidence`	Feed ↔ ZIP geographic coverage (218k+ mappings)
ContentPolicy	`category`, `action`, `keywords[]`	Content filtering rules (hard_block / soft_filter / allow)
GeoState/County/City/Zip	Hierarchy	Geographic hierarchy (5 pilot states, 31k ZIPs)
GeoCityZip	`cityId`, `zipId`	Many-to-many city ↔ ZIP
FeedAudit / ItemAudit	Audit trail	Every content/feed decision logged

Auth / Misc Models

Model	Purpose
PasswordResetToken	Password reset flow
UserFeedPreference	National RSS feed opt-in by industry

5.2 Tombstone (PostgreSQL — raw SQL, no ORM)

Table	Key Columns	Purpose
missions	`id`, `name`, `status`, `created_at`	Top-level mission records
tasks	`id`, `mission`, `department`, `summary`, `status`, `workflow_id`, `step_order`, `depends_on_task_id`, `input_from_task_id`, `claimed_by`, `claim_token`, `heartbeat_at`, `blocked_reason`	Central task queue
task_outputs	`id`, `task_id`, `agent`, `output` (JSON text), `created_at`	Agent outputs stored as JSON
task_watchdog	`task_id`, `assigned_agent`, `first_update_deadline`, `hard_timeout_deadline`, `watchdog_status`	Timeout monitoring
task_attachments	`task_id`, attachment fields	File attachments
scheduled_posts	`id`, `task_id`, `platform`, `content`, `scheduled_time`, `status`	Social posting queue (for publish_worker)
agent_heartbeats	`agent_name`, `department`, `status`, `task_id`, `last_beat`	Agent health monitoring
model_usage	LLM usage tracking	Token/cost tracking

Task Status Flow

```
Ready for Pickup → In Progress → Complete
                                → Failed (→ retry → Ready for Pickup)
Blocked → (dependency completes) → Ready for Pickup
```

---

6. RSS Intelligence System

Overview

- **462 feeds** (155 active) across 5 pilot states (CO, TX, FL, NC, MT)

- Types: local_news (190), weather/NWS (5 active), county gov (40 active), NPR stations (9 active), community
- **31,273 ZIPs** with city/county/state hierarchy
- **2,360 items** parsed, 96.4% approved by content policy

Clark Kent Scout Flow

1. `POST /api/rss/clark-kent` receives `{ zip, radius, websiteUrl, analysisId }`
2. Gathers RSS brief via `generateContentBrief(zip, radius)` — queries trade area feeds/items
3. Gathers upcoming events via `lib/social/upcoming-events.ts`
4. Returns `ScoutBrief` with `tradeArea`, `rssBrief`, `upcomingEvents`, `scoutSummary`
5. Scout summary sent to Tombstone as embedded context in social mission command

Content Policy Engine

- **Hard block:** sexual/adult, extreme violence
- **Soft filter:** political opinion, drugs/gambling, moderate violence
- **Auto-approve:** safe local content
- Keyword-based matching with ItemAudit trail

Trade Area Query

- ZIP → radius expansion → find overlapping FeedGeo entries → filter active feeds → approved items → rank by freshness + quality + diversity
- `getTradeAreaItems()`, `getItemsByRadius()`, `generateContentBrief()`

---

7. Image Storage & Resolution

Storage Locations

Source	Storage	URL Pattern	Access
GPT-5.1 generated ads	**AWS S3**	`https://{bucket}.s3.{region}.amazonaws.com/{key}`	Public
Tombstone agent artifacts	**Cloudflare R2**	R2 keys (not direct URLs)	Presigned URL via `/artifacts/resolve`
Social post images	**R2** (via Tombstone)	Proxied through `/api/social/image-proxy`	Server-side proxy

Image Resolution Chain

1. **S3 URLs** (contain `.s3.` + `amazonaws.com`) → pass through directly
2. **R2 keys** → resolve via Tombstone `/artifacts/resolve` endpoint → presigned URL
3. **Data URLs** (`data:image/...`) → pass through directly

4. **Social images** → `/api/social/image-proxy?key=...` → server fetches from Tombstone, streams bytes

Why the proxy?

- R2 presigned URLs expire and are unreliable (403 on HEAD, 200 on GET)
- Cloudflare R2 sometimes returns 403 unpredictably
- Server-side proxy with 24h cache headers solves this

---

8. Authentication & CRM

Auth

- **NextAuth** with credentials provider (email + password)
- Business email only — blocks Gmail, Yahoo, Hotmail, etc. (`lib/email-validation.ts`)
- JWT sessions, no database sessions
- Email confirmation flow via Abacus notification API
- Password reset flow via Abacus notification API

CRM (GoHighLevel)

- `createGHLContact()` — creates CRM contact on registration
- `sendGHLEmail()` — email via GHL (legacy, replaced by Abacus notifications for transactional)
- `sendConfirmationEmail()` — legacy, now uses Abacus notification system

---

9. Environment Variables

Ad Launch (`nextjs_space/.env`)

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string (Prisma)
`NEXTAUTH_SECRET`	NextAuth JWT signing secret
`TOMBSTONE_API_URL`	Tombstone FastAPI base URL (Render)
`OPENAI_API_KEY`	OpenAI API key (GPT-5.1 ad generation, ad editing)
`ABACUSAI_API_KEY`	Abacus AI API key (LLM routing, notifications)
`AWS_PROFILE`	AWS profile name for S3
`AWS_REGION`	AWS region for S3 bucket
`AWS_BUCKET_NAME`	S3 bucket for ad images
`AWS_FOLDER_PREFIX`	S3 key prefix for ad images
`GOOGLE_MAPS_API_KEY`	Google Places Text Search API
`GHL_API_TOKEN`	GoHighLevel API token
`GHL_LOCATION_ID`	GoHighLevel location ID
`WEB_APP_ID`	Abacus notification app ID
`NOTIF_ID_EMAIL_CONFIRMATION`	Notification type ID for email confirmation
`NOTIF_ID_PASSWORD_RESET`	Notification type ID for password reset
`NEXTAUTH_URL`	Auto-configured by Abacus AI per environment

Tombstone (`.env` on Render)

Variable	Purpose
`DATABASE_URL`	Tombstone PostgreSQL connection string
`OPENAI_API_KEY`	OpenAI API key (agent LLM calls — GPT-5.1)
`OPENAI_MODEL`	Default model (gpt-5.1)
`LLM_PROVIDER`	Default LLM provider (`openai` on Render)
`OLLAMA_BASE_URL`	Local Ollama URL (dev only)
`R2_ENDPOINT` / `R2_BUCKET` / `R2_ACCESS_KEY` / `R2_SECRET_KEY`	Cloudflare R2 storage
`TELEGRAM_BOT_TOKEN`	Telegram bot for admin notifications
`WP_BASE_URL` / `WP_USERNAME` / `WP_APP_PASSWORD`	WordPress integration (legacy)
`USE_POSTGRES`	Flag to use PostgreSQL (always true in production)
`TASK_HEARTBEAT_TIMEOUT_SECONDS`	Heartbeat timeout for agents
`GEORGE_EXECUTION_BACKEND`	Code execution backend for George Boole

---

10. Deployment & Runtime

Ad Launch

- **Hosted on:** Abacus AI platform
- **Framework:** Next.js 14 (App Router), standalone output mode
- **Build:** `yarn run build` → `.build/standalone` → deployed as tarball
- **Database:** PostgreSQL (shared dev/prod via Prisma)
- **Package manager:** yarn only (no npm/npx)
- **Custom domain:** `connect.launchmarketing.com`
- **Abacus subdomain:** `ad-launch-1nfyr8.abacusai.app`

Tombstone OS

- **Hosted on:** Render (web service)
- **Framework:** FastAPI (Python 3.12)
- **Runtime:** Single process, all agents as threads (`run/thread_runner.py`)
- **Database:** PostgreSQL (separate from Ad Launch DB)
- **Cold starts:** Render free tier — 10-15 second cold start when idle

- **Process model:**
- 7 pipeline agent threads (Bridger, Zig, Ogilvy, Draper, Warhol, Drucker, Boole)
- 3 service threads (Dispatcher, Watchdog, Wyatt)
- FastAPI server (main thread)
- Publish Worker runs separately (not in thread runner)

Inter-Service Communication

```
Ad Launch ──HTTP──► Tombstone (TOMBSTONE_API_URL)
  POST /commands         → Create missions/workflows
  GET  /tasks            → Poll task status
  GET  /tasks/{id}/outputs → Fetch agent outputs
  GET  /artifacts/resolve → Resolve R2 presigned URLs
  GET  /content/queue    → Social content queue
  GET  /content/{id}     → Content detail
```

---

11. Known Issues & Gotchas

R2 Presigned URLs

- Cloudflare R2 returns 403 on HEAD requests but 200 on GET
- Presigned URLs expire unpredictably
- **Solution:** Server-side image proxy (`/api/social/image-proxy`)

Tombstone Cold Starts

- Render free tier spins down after inactivity (10-15s cold start)
- Images appear blank during cold start
- Loading spinners added to handle this

Ad Duplication Race Condition (Fixed)

- `mission-status` used `status: { not: 'completed' }` for atomic lock
- Multiple concurrent polls could all pass the check
- **Fix:** Changed to `status: { notIn: ['completed', 'completing'] }`

Cloudflare Bot Protection (Fixed)

- Sites behind Cloudflare return 403 with challenge page
- Previously treated as “unreachable”
- **Fix:** Detect `cf-mitigated: challenge` header → treat as reachable, skip HTML scraping, fall back to Google Places

Content Queue vs Workflow Tasks

- Tombstone `/content/queue?workflow_ids=...` returns individual render tasks (not workflow-level)
- Detail endpoint `/content/{task_id}` has `base_caption` , `cta` , `hashtags` from upstream task walking
- Social polling in Ad Launch uses content queue (not `getSocialWorkflowResults()` which required ALL tasks complete)

Downloads in iframe

- Must use blob + createObjectURL + `<a>` click pattern

- `window.open()` doesn't trigger downloads from within preview iframe

---

12. Future / Unfinished

Auto-Posting API Integration

- Publish Queue page has "Coming Soon" overlay
- Social publishers are stubs — no real API calls
- **Facebook/Instagram:** Requires Meta App Review for `publish_pages` scope
- **YouTube:** Google API console + OAuth consent review
- **Pinterest:** Partner-level API access
- **TikTok/Snapchat:** No public posting API exists

Social Post Image Generation (Tombstone-side)

- Clark Kent currently produces text-only posts
- Andy Warhol generates images for ads but social post images come from Tombstone R2
- Next step: GPT-5.1 generation for each social post with social-optimized prompts

Courthouse Address for County Geo-Tagging

- County gov feeds fall back to state-level ZIP assignment
- Solution: extract courthouse address from county website during discovery

RSS Expansion

- Currently 5 pilot states (CO, TX, FL, NC, MT)
- Need to expand to all 50 states
- NPR station coverage is thin (many use JS rendering, no static RSS)

Payment / Monetization

- Currently free tier only (watermarked ads, `freeAdsUsed` counter)
- `paidAdsCount` field exists but no payment integration
- No Stripe or payment flow implemented

Appendix: File Tree (Key Files Only)

```
ad_launch/
├── ARCHITECTURE.md                ← This file
├── .project_instructions.md       ← Agent memory (design decisions, feature history)
└── nextjs_space/
    ├── app/
    │   ├── page.tsx               ← Landing page (Social Posting Factory)
    │   ├── layout.tsx             ← Root layout, metadata, providers
    │   ├── globals.css            ← CSS variables, animations
    │   ├── providers.tsx          ← SessionProvider wrapper
    │   ├── api/                   ← All API routes (see §3.2)
    │   ├── analyze/[id]/          ← Analysis tracker page
    │   ├── results/[id]/          ← Results page
    │   ├── dashboard/             ← Business dashboard, social, publishing, feeds
    │   ├── admin/rss/             ← RSS admin dashboard
    │   ├── components/            ← Shared components (header, watermark-card, etc.)
    │   └── (auth pages)           ← login, confirm, reset-password
    ├── lib/                       ← Business logic modules (see §3.3)
    │   ├── tombstone.ts           ← Tombstone API client
    │   ├── generate-ad-image.ts   ← GPT-5.1 ad generation
    │   ├── rss/                   ← RSS intelligence subsystem
    │   └── social/                ← Social subsystem
    ├── prisma/
    │   └── schema.prisma          ← Data models (see §5.1)
    └── .env                       ← Environment variables

tombstone/
├── agents/                        ← All agents (see §4.1)
│   ├── jim_bridger.py
│   ├── zig_ziglar.py
│   ├── david_ogilvy.py
│   ├── don_draper.py
│   ├── andy_warhol.py
│   ├── wyatt.py                   ← Command router
│   ├── dispatcher.py              ← Task monitoring
│   └── task_watchdog.py           ← Timeout enforcement
├── core/                          ← Infrastructure (see §4.4)
│   ├── task_service.py            ← Mission/task CRUD
│   ├── task_reliability.py        ← Claims, heartbeats, retry
│   ├── model_router.py            ← LLM routing
│   ├── r2_storage.py              ← R2 client
│   └── pg_runtime.py              ← PostgreSQL client
├── server/
│   └── mission_control_api.py     ← FastAPI server (see §4.5)
├── services/social_publishers/    ← Stub publishers
├── workers/
│   └── publish_worker.py          ← Social posting queue processor
├── run/
│   ├── thread_runner.py           ← Single-process thread runner
│   └── start_worker.sh            ← Render entrypoint
├── config/                        ← Settings, env loader
├── tools/                         ← Utility tools (image gen, SEO, etc.)
└── migrations/                    ← SQL migrations
```